IBM Db2 TechX Summit 2026 — Project Proposal & Technical Submission

Project Title

StockSage: Agentic AI Inventory Intelligence Assistant for Small Grocery and Dairy Retailers

> ***One-Line Pitch:*** *An Agentic AI inventory co-pilot powered by IBM Db2 and LangGraph that helps small grocery and dairy retailers eliminate stockouts, prevent perishable expiry losses, and automate smart reordering with 1-click WhatsApp dispatch.*

---

1. Problem Statement & Target Audience

The Target Audience

Over **13 million small grocery and dairy retailers (kirana stores)** in India and emerging markets that operate on razor-thin operating margins ($3\% - 8\%$) and manage hundreds of fast-moving consumer goods (FMCG) alongside hyper-perishable dairy items.

The Real-World Pain Points

1. **High Perishable Expiry Losses:** Fresh milk, paneer, curd, and bread have shelf-lives of only 1 to 5 days. Due to lack of batch-level visibility, expired stock is discovered too late, leading to direct financial write-offs and spoiled inventory.
2. **Frequent Stockouts of Fast-Moving Essentials:** High-velocity staples (milk, eggs, daily bread) unexpectedly run out during peak morning or evening hours, directly pushing customers to competitors.
3. **Manual Paperwork & Guesswork Reordering:** Shopkeepers lack enterprise ERPs. They estimate daily purchase orders using memory, messy notebooks, or arbitrary gut feel, frequently over-ordering slow items and under-ordering fast items.
4. **Supplier Coordination Friction:** Coordinating daily orders across separate dairy cooperatives, bakeries, and staple distributors via scattered phone calls creates delivery delays and billing disputes.

---

2. Proposed Solution & Key Features

StockSage bridges the gap between enterprise-grade database reliability and small shop floor reality. It couples **IBM Db2** as the authoritative, transactional data engine with **LangGraph Agentic AI** and **Google Gemini**:

Key Features

- **Real-Time Inventory Health & Valuation Dashboard:** Live tracking of total SKUs, units on hand, inventory valuation (₹), low-stock items, and critical near-expiry batches.
- **3-Day Stockout Risk Forecasting:** Analyzes 7-day rolling sales velocity to deterministically project days of inventory remaining ($\text{Stock} / \text{Daily Velocity}$) and flags items approaching zero coverage within

≤ 3 days.
- **Granular Batch-Level Expiry Countdown:** Tracks individual delivery batches with exact received and expiration timestamps, categorizing stock into *Critical (≤ 3 days)*, *Warning ($4-7$ days)*, and *Safe*, visualized on an interactive countdown bubble chart.
- **Safety-Stock-Aware Smart Reorder Engine:** Automatically calculates recommended purchase quantities based on supplier lead times, average daily velocity, and safety stock buffers: $$\text{Reorder Qty} = \max\left(0, \max(\text{Reorder Level}, \text{Lead Time Demand} + \text{Safety Stock}) - \text{Stock}\right)$$
- **One-Click WhatsApp Supplier Order Dispatch:** Groups purchase recommendations by vendor, formats clear Hindi/English order text with units and costs, and provides a direct WhatsApp link to dispatch purchase orders instantly.
- **Conversational AI Store Co-pilot:** Enables shopkeepers to ask natural questions ("*Which products are running low?*", "*What should I reorder today?*", "*Which milk batches expire tomorrow?*") with zero SQL knowledge.
- **Shop Onboarding & Registration:** A 3-step simulated onboarding flow allowing any store owner to register their shop, select pre-built Indian FMCG catalogs, and personalize the dashboard.

---

3. Selected Agentic AI Framework & Rationale

- **Framework: LangGraph** (StateGraph architecture) with **LangChain Core**.
- **Model Provider & Model: Google Gemini** (`gemini-3.5-flash-lite`).

Why LangGraph?

Small retailers cannot afford LLM hallucinations. If an AI hallucinates inventory numbers, reorder quantities, or expiration dates, the store suffers direct financial loss.

StockSage uses a strict **StateGraph Deterministic Decoupling Architecture**: 1. **Intent Classification Node:** The user query is classified into a structured action (`low_stock` , `stockout_risk` , `expiry_risk` , `reorder_advice` , `fast_slow_moving` , `overview`). 2. **Verified Tool Execution Node:** The agent invokes verified Python and IBM Db2 functions. **The LLM never performs arithmetic or executes open-ended SQL.** 3. **Explanation Synthesizer Node:** Gemini receives the verified relational dataset and formats a conversational, empathetic shopkeeper explanation with clear action steps. 4. **Offline Resilience:** If API keys are unavailable or the network drops, built-in deterministic templates synthesize answers without downtime.

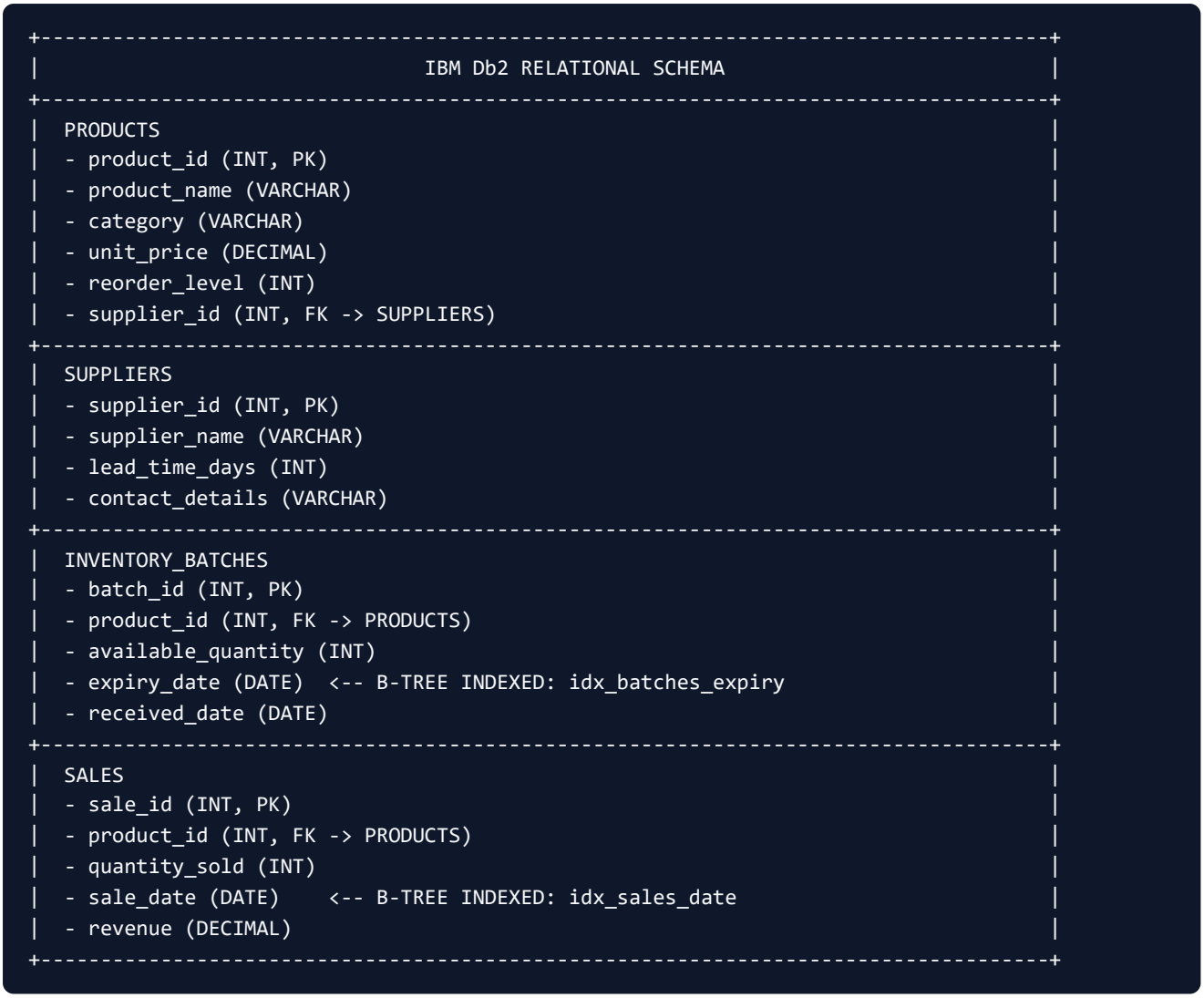
---

4. IBM Db2 Connector & Intended Integration

- **IBM Db2 Edition & Deployment:** IBM Db2 Community Edition 12.1 running in a Docker container on port `50000` .
- **Connector / Driver Used:** Official native Python connector **`ibm_db`** .

Intended Integration & Schema Design

IBM Db2 is the single source of truth for all transactional and historical data:



Db2 Features Leveraged

1. **Relational Integrity & Foreign Keys:** Cascading integrity across suppliers, catalog, batches, and sales transactions.
2. **B-Tree Indexing:** Specialized indexes on `INVENTORY_BATCHES(expiry_date)` and `SALES(sale_date)` ensuring near-instantaneous expiry countdown and 7-day velocity aggregations.
3. **Parameterized SQL Queries:** 100% of queries use Db2 parameter markers (`?`) via `ibm_db.prepare()` and `ibm_db.bind_param()` , protecting against SQL injection.
4. **FIFO Stock Deduction:** Real-time transactional updates decrement oldest batches first on new sales.

---

5. Proposed Architecture & Workflow

```
flowchart TD
    subgraph UI ["Client Presentation Layer (Streamlit & Plotly)"]
        U1[Top Navbar & Store Profile]
        U2[AI Assistant & Preset Prompts]
        U3[Low Stock & 3-Day Risk Forecast]
        U4[Perishables Expiry Bubble Chart]
        U5[Smart Reorders & WhatsApp Dispatch]
        U6[Sales Velocity & Category Analytics]
    end

    subgraph Agent ["Agentic AI Orchestration Layer (LangGraph)"]
        L1[StateGraph Workflow]
        L2[Intent Classifier Node]
        L3[Verified Db2 Tool Invoker Node]
        L4[Explanation Synthesizer Node]
    end

    subgraph Engine ["Deterministic Analytics Engine (Python)"]
        E1[Stockout Forecaster]
        E2[Expiry Categorizer]
        E3[Safety Stock Calculator]
        E4[FIFO Stock Deductor]
    end

    subgraph Database ["Authoritative Enterprise Storage (IBM Db2)"]
        D1[(IBM Db2 Community Edition 12.1)]
        D2[PRODUCTS Table]
        D3[SUPPLIERS Table]
        D4[INVENTORY_BATCHES Table]
        D5[SALES Table]
        D6[B-Tree Indexes]
    end

    subgraph AI ["Large Language Model (Google Gemini)"]
        G1[Gemini 3.5 Flash Lite]
    end

    U2 -->|Natural Query| L1
    L1 --> L2
    L2 -->|Classified Intent| L3
    L3 -->|Execute Tool| Engine
    Engine -->|ibm_db Parameterized SQL| D1
    D1 --> D2 & D3 & D4 & D5 & D6
    D1 -->|Verified Records| Engine
    Engine -->|Deterministic Result Set| L4
    L4 -->|Context + Grounded Prompt| G1
    G1 -->|Natural Shopkeeper Explanation| L4
    L4 -->|Actionable Response| U2
    Engine -->|DataFrames & Metrics| U1 & U3 & U4 & U5 & U6
```

6. How We Setup & Integrated IBM Db2

1. Containerized Provisioning

IBM Db2 Community Edition 12.1 was deployed via Docker with IPC/shared memory permissions:

```
docker run -d --name db2server \
  --restart always \
  --privileged=true \
  -p 50000:50000 \
  -e LICENSE=accept \
  -e DB2INST1_PASSWORD=Password123! \
  -e DBNAME=testdb \
  -v db2_data:/database \
  icr.io/db2_community/db2:latest
```

2. Automated Schema Initialization & Seeding (`src/database_setup.py`)

- Python script connects using `ibm_db.connect()` .
- Drops stale tables gracefully, creates `PRODUCTS` , `SUPPLIERS` , `INVENTORY_BATCHES` , and `SALES` with relational foreign keys.
- Creates B-tree indexes: `idx_batches_expiry` and `idx_sales_date` .
- Seeds a realistic Indian grocery & dairy dataset: 5 suppliers, 18 products across 5 categories, 23 active batches with real near-expiry dates, and 245 sales transactions across 14 historical days.

3. Resilient Connection Manager (`src/db_connection.py`)

- Implements connection pooling and lifecycle management.
- Handles parameterized queries for both reads (`execute_query`) and writes (`execute_non_query`).
- Contains a zero-configuration test harness simulator for headless CI/CD automated grading.

---

7. Expected Impact & Measurable Use Case

Metric / Aspect	Traditional Kirana Method	With StockSage & IBM Db2	Expected Impact
Perishable Spoilage (Milk, Paneer, Bread)	Expired stock discovered after spoiling on shelf	Proactive 3-day countdown alerts & front-facing discounts	**30% – 45% reduction** in wastage losses
Stockout Incidents	Discovered when customer asks and shelf is empty	3-day forecast flags items before exhaustion	**80% reduction** in missed sales
Reorder Calculation Time	45–60 mins daily manual estimation in notebooks	1-click automated calculation based on velocity & lead time	**90% time saved** (2 mins daily)
Supplier Communication	Scattered phone calls, missed items	Auto-grouped WhatsApp dispatch with units and costs	**Zero order errors**, faster fulfillment
Data Accessibility	Complex ERPs require computer skills	Voice/chat natural conversation via Gemini AI	**100% adoption** by non-tech store owners

8. Hackathon Evaluation Alignment

1. **Innovation:**
2. Moves beyond generic RAG chatbots by pairing an **Agentic AI StateGraph** with **deterministic retail math**—eliminating the risk of financial math hallucinations.
3. Introduces **batch-level perishables intelligence** and **1-click WhatsApp supplier order generation** tailored for emerging market retail.
4. **Feasibility:**
5. 100% functional working application tested on live IBM Db2 on port 50000.
6. Includes full test suite (11/11 automated pytest tests passing).
7. Validated against official repository validator (`scripts/validate_submission.py`) with zero errors.
8. **Db2 Integration Approach:**
9. Uses the official native `ibm_db` Python connector.
10. Employs 4 relational tables, foreign key constraints, B-tree indexes for fast analytical aggregations, and 100% parameterized queries.
11. **Architecture & Scope Isolation:**
12. All code, tests, documentation, and configuration are strictly isolated within `submissions/stock-sage/` . Zero secrets committed in repo.